

# 1) You upgraded Angular from a lower version to a higher version. What steps did you follow? Did you follow any official documentation during the upgrade?

Yes.

I always follow Angular's official migration process rather than directly changing package versions.

## Step 1 - Check current version

```
<> Bash  
  
ng version
```



## Step 2 - Backup

- Commit code
- Create Git branch

```
git checkout -b angular19-upgrade
```



## Step 3 - Check Angular Update Guide

Angular provides an official migration guide.

I select

- From Version
- To Version
- Complexity

It provides all required steps.

## Step 4 - Update CLI

```
ng update @angular/cli
```



---

## Step 5 - Update Core

```
ng update @angular/core
```



---

## Step 6 - Update Angular Material

```
ng update @angular/material
```



---

## Step 7 - Update Third-party Packages

Example

```
npm outdated
```

```
npm install rxjs@latest
```

```
npm install ngrx@latest
```



---

## Step 8 - Resolve Breaking Changes

Examples

- Deprecated APIs
- Standalone migration
- Signals
- Hydration
- SSR changes
- Zone-less changes

---

## Step 9 - Run Tests

ng test



ng lint

ng e2e

---

## Step 10 - Performance Testing

- Lighthouse
- Bundle analysis
- Core Web Vitals

---

## Step 11 - Deploy to QA

Smoke testing

Regression testing

Accessibility testing

---

## 2) Which documentation or migration guide did you use?

Main references:

- Angular Update Guide
- Angular Release Notes
- Angular Blog
- Angular GitHub Releases

Also,

- RxJS migration guide
- NgRx migration guide
- Angular Material migration guide

---

## 3) What was one unique challenge you faced during the Angular upgrade, and how did you solve it?

### Example (Angular 16 → Angular 19)

#### Challenge:

Webpack Builder was replaced by the newer build system.

#### Issues:

- Custom webpack stopped working.
- Some plugins became incompatible.
- Lazy loading broke.

#### Solution:

- Migrated to the new Angular build system.
- Removed unnecessary webpack customization.
- Replaced deprecated APIs.
- Updated lazy route syntax.
- Updated SSR configuration.
- Updated esbuild configuration.

#### Another challenge:

Standalone migration.

#### Earlier:

AppModule



#### Later

bootstrapApplication()



We migrated gradually module-by-module instead of rewriting the entire application.

---

## 4) What is the purpose of package.json? What is package-lock.json?

### package.json

It is the project manifest.

## Contains

- Project name
- Scripts
- Dependencies
- Dev dependencies
- Version
- Build commands

## Example

<> JSON



```
{
  "scripts": {
    "start": "ng serve",
    "build": "ng build"
  }
}
```

---

## package-lock.json

Locks the exact dependency tree.

Without lock file

Developer A

rxjs 7.8.0



Developer B

rxjs 7.8.2



Application may behave differently.

Lock file guarantees

Everyone installs

Exactly

rxjs 7.8.0



## 5) When you run npm install, what exactly happens internally?

Flow

npm install



↓

Read package.json

↓

Read package-lock.json

↓

Resolve dependencies

↓

Download packages

↓

Download transitive dependencies

↓

Verify integrity

↓

Execute lifecycle scripts

↓

Create node\_modules

↓

```
Update package-lock.json
```

Internally

- Dependency resolution
- Version conflict resolution
- Registry communication
- Integrity verification (SHA)
- Package extraction
- Binary linking

## 6) What is an npm registry?

An npm registry is a package repository.

Default

```
https://registry.npmjs.org/
```



It stores

- Angular
- React
- Express
- RxJS

Everything is downloaded from the registry.

## 7) How would you install a package that is not available in the public npm registry?

Several options.

**Private npm Registry**

```
npm install @company/logger
```



Registry configured as

```
https://npm.company.com
```



---

## Git Repository

```
npm install git+https://github.com/company/logger.git
```



---

## Local Folder

```
npm install ../logger
```



---

## Tar File

```
npm install logger-1.0.0.tgz
```



---

## GitHub Package Registry

```
npm install @company/logger
```



after authentication.

---

**8) Suppose an organization has 10 Angular applications requiring the same logging functionality. Would you implement it separately?**

No.

Never duplicate code.

I'd build a reusable Angular library.

Architecture

Apps



App1

App2

App3

↓

Common Logging Library

↓

Publish

↓

Version

↓

Reuse

### Benefits

- Single source of truth
- Easier maintenance
- Consistent logging
- Independent versioning

## 9) How would you build and distribute a reusable logging library?

Create library

```
ng generate library logging
```



Structure

```
projects/
```



```
logging/
```

```
logger.service.ts
```

```
logger.module.ts
```

```
public-api.ts
```

## Service

<> TypeScript



```
@Injectable({  
  providedIn: 'root'  
})  
export class LoggerService {  
  
  log(message:string){  
  
    console.log(message);  
  
  }  
  
}
```

## Build

```
ng build logging
```



## Publish

```
npm publish
```



or

## Publish into

- Nexus
- Artifactory
- Azure Artifacts
- GitHub Packages
- Verdaccio (internal npm registry)

## Install

```
npm install @company/logging
```



## Challenges in Micro Frontends

Common challenges:

### Shared dependencies

Avoid loading Angular multiple times using Module Federation shared configuration.

### Version mismatch

Different MFEs using Angular 17 vs Angular 19.

Mitigation:

- Align framework versions where possible.
- Use shared singletons for Angular runtime.

### State synchronization

Different MFEs require shared authentication/user state.

Solution

- Shared Auth Service
- Event Bus
- RxJS Subjects
- NgRx
- Custom events

### Routing

Shell owns global routing.

Remote owns feature routing.

### Authentication

Centralized OAuth provider.

### CSS conflicts

Shadow DOM

or

CSS prefixing.

## Deployment

Independent deployment.

Semantic versioning.

---

## 10) Can Angular be used only for frontend applications?

Primarily yes, but Angular supports more than browser SPAs:

- Single Page Applications (SPA)
- Server-Side Rendering (SSR)
- Static Site Generation (SSG/Prerender)
- Progressive Web Apps (PWA)
- Hybrid rendering
- Desktop applications using Electron
- Mobile applications using NativeScript or Ionic
- Micro Frontends
- Web Components (Angular Elements)

Angular is not used to build backend APIs directly.

---

## 11) How would you manage state in a Micro Frontend architecture?

Keep state ownership clear.

Global State

Authentication

User

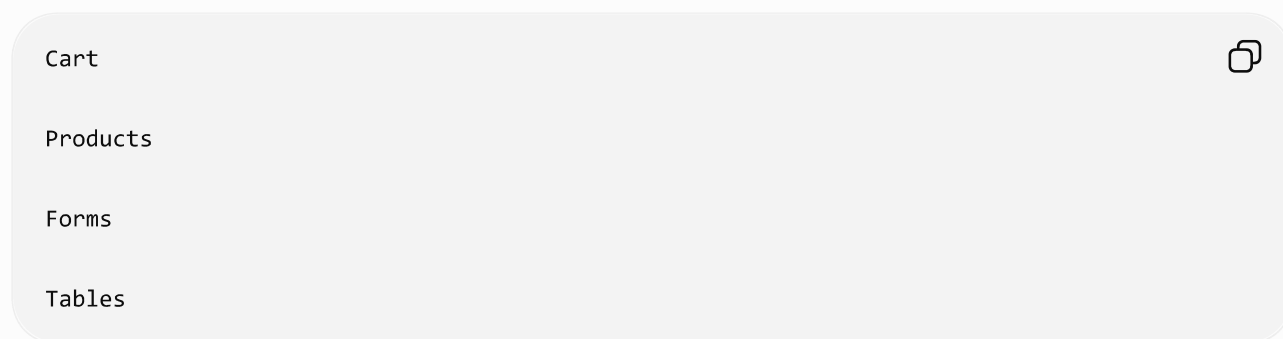
Theme

Language



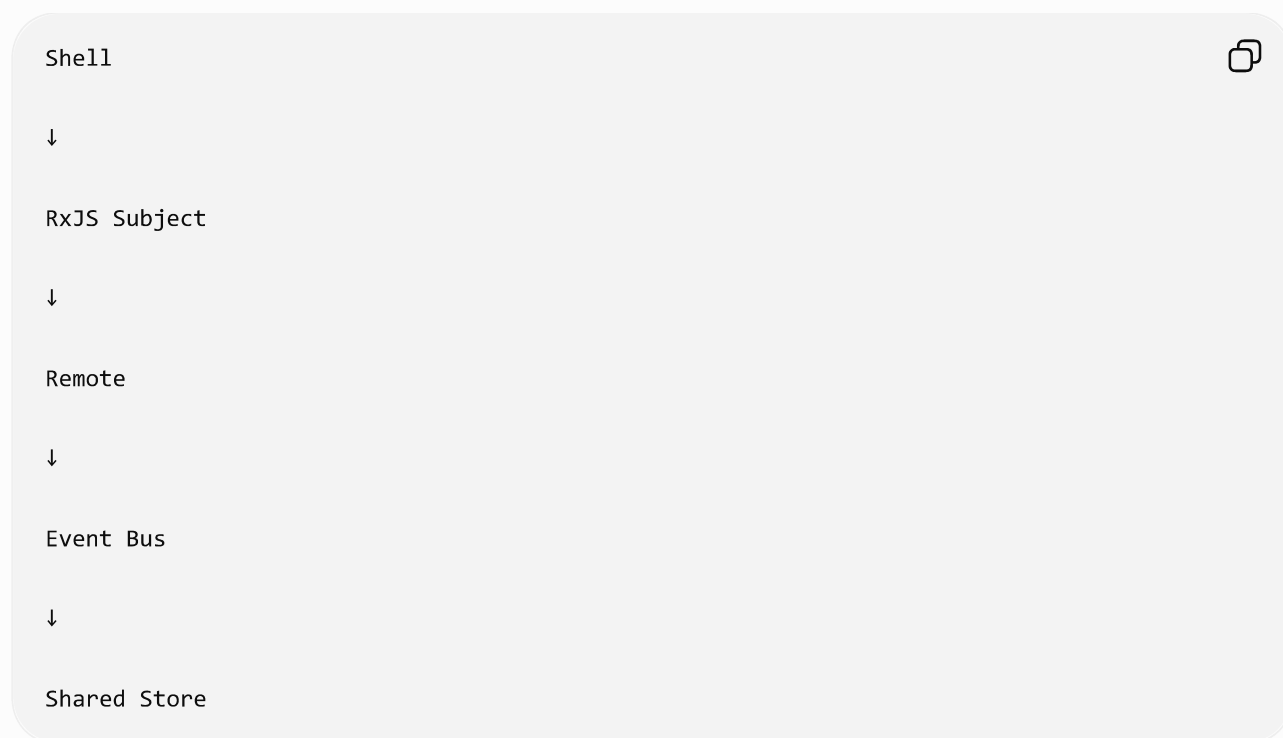
Managed in Shell.

Local State



Managed inside each MFE.

Communication



Avoid one giant global store shared across all MFEs because it increases coupling.

---

## 12) How do different Angular applications share or transfer data?

Methods include:

- REST APIs
- GraphQL
- WebSockets

- Custom Browser Events ( `CustomEvent` )
- RxJS Subjects (when sharing runtime)
- Module Federation shared services
- Shared libraries
- Browser storage (limited use cases)
- `postMessage` for iframe communication

## 13) How does data flow in NgRx?

Component



↓

`dispatch(Action)`

↓

Reducer

↓

Store

↓

Selector

↓

Component

If asynchronous work is needed:

Component

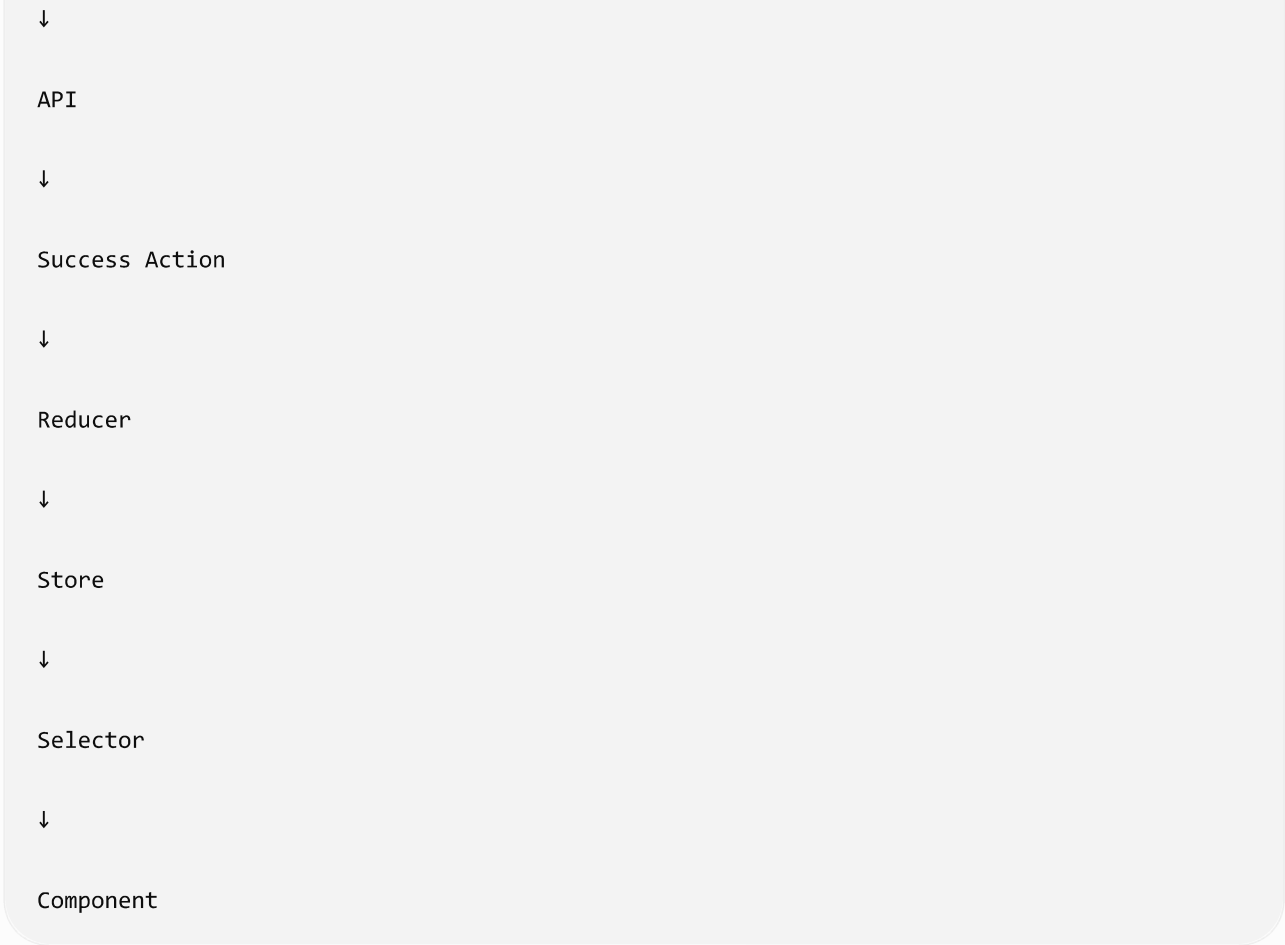


↓

Action

↓

Effect



The component never updates state directly.

## 14) Is NgRx the only option for state management?

No. Other options include:

| Solution             | Best for                                     |
|----------------------|----------------------------------------------|
| Signals              | Local component state                        |
| SignalStore          | Medium to large Angular applications         |
| NgRx Store           | Large enterprise apps with predictable state |
| NGXS                 | Simpler Redux-style state                    |
| Akita                | Entity-heavy applications                    |
| RxJS BehaviorSubject | Small applications or feature state          |

| Solution              | Best for            |
|-----------------------|---------------------|
| ComponentStore        | Local feature state |
| Services with Signals | Simple shared state |

Choose the simplest solution that meets the application's complexity.

## 15) How can data be transferred from one project to another?

Depending on architecture:

- Shared Angular library
- REST API
- GraphQL
- Message queues (Kafka, RabbitMQ) via backend
- WebSockets
- Module Federation shared services
- Browser `CustomEvent`
- `postMessage` (cross-window/iframe)
- IndexedDB or local storage for offline scenarios (not for sensitive data)

## 16) How can reusable business logic or common code be shared between multiple Angular projects?

Create a shared library containing:

- Services
- Utilities
- Models
- Pipes
- Directives
- Validators
- Interceptors
- Guards
- UI components

- Business rules

Monorepos (e.g., Nx) are especially useful because libraries can be versioned and reused efficiently within the workspace.

---

## 17) Can you explain with code how you would create and consume a shared Angular library?

### Step 1: Create the library

</> Bash



```
ng generate library shared-lib
```

### Step 2: Create a service

</> TypeScript



```
// projects/shared-lib/src/lib/logger.service.ts

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class LoggerService {
  log(message: string): void {
    console.log(`[LOG] ${message}`);
  }

  error(message: string): void {
    console.error(`[ERROR] ${message}`);
  }
}
```

### Step 3: Export it

&lt;/&gt; TypeScript



// public-api.ts

export \* from './lib/logger.service';

---

## Step 4: Build the library

&lt;/&gt; Bash



ng build shared-lib

---

## Step 5: Publish (or reference locally)

&lt;/&gt; Bash



npm publish

or install the generated package from a private registry or local tarball.

---

## Step 6: Consume in another Angular application

&lt;/&gt; TypeScript



```
import { Component, inject } from '@angular/core';
import { LoggerService } from '@company/shared-lib';

@Component({
  selector: 'app-home',
  standalone: true,
  template: `<button (click)="save()">Save</button>`
})
export class HomeComponent {
  private logger = inject(LoggerService);

  save(): void {
    this.logger.log('Save button clicked');
  }
}
```

```
}  
}
```

This approach centralizes common functionality, enables semantic versioning, reduces duplication, and allows fixes or enhancements to be rolled out across all Angular applications by updating the library version.

